Equity Research Prototype — Project Guide

Local-first semantic equity research over 100 NYSE companies. FastAPI + Postgres/pgvector + Next.js, all models local via Ollama. Retrieval & cited research only — no investment advice.

1. Directory structure — what lives where

```
easy-invest/
├─ SPEC.md                    The build specification this repo implements
├─ README.md                  Setup & run instructions
├─ docker-compose.yml         Postgres + pgvector (db), optional api/web containers
├─ .env.example               All config: DB URL, Ollama models, SEC user-agent
├─ data/
│  ├─ companies.csv           The FIXED universe: 100 reviewed NYSE companies (ticker,name,cik,…)
│  ├─ eval_queries.json       25 labeled queries for retrieval evaluation
│  ├─ filings/  (gitignored)  Downloaded SEC filing HTML, cached forever by accession no.
│  └─ cache/    (gitignored)  Cached SEC JSON, market snapshots, query plans
├─ scripts/
│  ├─ build_universe.py       Regenerates companies.csv, validated against SEC data
│  ├─ bootstrap_db.py         Runs migrations + loads the universe into Postgres
│  ├─ ingest_universe.py      Pulls filings metadata, 10-Ks, XBRL facts, market caps
│  ├─ rebuild_cards.py        Builds the semantic card index (the long LLM job)
│  ├─ card_progress.py        Live progress bar for the card build
│  └─ evaluate_search.py      Recall/precision/latency report over eval_queries.json
├─ services/api/              ── PYTHON BACKEND ──
│  ├─ alembic/                DB migrations (13 tables, vector indexes)
│  ├─ tests/                  63 unit + integration tests
│  └─ app/
│     ├─ main.py              FastAPI entrypoint
│     ├─ core/                config.py, db.py, llm.py (Ollama provider), cache.py, logging.py
│     ├─ models/tables.py     All SQLAlchemy tables (companies, cards, filings, sessions…)
│     ├─ schemas/             Pydantic: SearchPlan, conditions, results, API bodies
│     ├─ prompts/             Every LLM prompt: cards, planner, catalyst, chat
│     ├─ api/                 Routers: search.py, companies.py, admin.py
│     ├─ workers/             candidate_worker.py — bounded per-company research
│     └─ services/
│        ├─ sec/              EDGAR client (rate-limited), HTML parser, XBRL extraction
│        ├─ market_data/      yfinance snapshots (12h cache, labeled delayed)
│        ├─ semantic_search/  cards.py (generation), retrieval.py, indexer.py
│        ├─ query_planner/    NL query → structured SearchPlan
│        ├─ research/         financials.py (deterministic YoY), catalysts.py
│        ├─ ranking/          filters.py (market cap etc.), scoring.py (final rank)
│        ├─ chat/             Follow-up: intent router, grounded answers, plan editor
│        ├─ search_service.py The orchestrator tying all stages together (SSE progress)
│        └─ ingestion.py      Universe/company ingestion pipeline
└─ apps/web/                  ── NEXT.JS FRONTEND ──
   ├─ app/                    Pages: / (search), search/[id], company/[ticker], admin
   ├─ components/             search-view, result-card, funnel, chat-panel, plan-chips…
   └─ lib/api.ts              Typed (Zod) API client + SSE stream helper
```

2. How the semantic search works

The core idea: “company cards”

You can’t embed a 300-page 10-K as one vector and expect good search. Instead, each company is distilled **offline** into ~25 small, verified facts called **cards** — e.g. *“Operates interconnected data centers providing colocation and connectivity to enterprises and cloud providers.”* Each card has a type (business_activity, product_service, customer_exposure, geographic_exposure, supply_chain_role, macro_exposure), a directness label (core / direct / indirect / prospective), and the exact 10-K excerpt it came from.

Offline: building the index (scripts/rebuild_cards.py)

1. Parse. The company's latest 10-K HTML is stripped and split into sections (Item 1 Business, Item 2 Properties, Item 7 MD&A), then chunked into ~1800-char excerpts.

2. Extract. Each excerpt goes to the local LLM (qwen3:8b) with strict rules: one atomic fact per card, 20–50 words, no opinions, no inference beyond the text.

3. Verify. A *second* LLM pass judges every candidate card against its source excerpt: entailed / partially / not entailed. Only **entailed cards with confidence ≥ 0.75** survive — this is what keeps the index trustworthy.

4. Embed. Surviving card texts become 1024-dim vectors (qwen3-embedding:0.6b) stored in Postgres with **pgvector**, alongside the source excerpt and filing URL for citations.

Online: answering a query

1. Plan. The LLM converts your sentence into a structured `SearchPlan`: semantic conditions (concepts to match, which card types, required directness), structured filters (sector, market-cap "around $3B ±40%"), and research conditions (revenue YoY ≥ X, recent catalyst). Shown as editable chips in the UI.

2. Retrieve. Each semantic condition is embedded and pgvector finds the top-100 nearest cards by cosine similarity, filtered by card type/directness. Cards group by company (best 3 kept). A company must clear a similarity floor (0.55) and have evidence for *every required* condition. Score = `0.70·similarity + 0.20·directness + 0.10·confidence`.

3. Filter. Deterministic structured filters (ticker/sector/market cap) cut the list to ≤15 candidates. Missing market data ⇒ "not verified", never a silent fail.

4. Research. One bounded worker per candidate runs in parallel (25s deadline, ≤4 downloads, ≤3 LLM calls). Revenue/net-income YoY is computed **in Python from SEC XBRL facts — never by the LLM** (same fiscal quarter one year apart, restatements preferred). Catalyst questions retrieve relevant chunks from recent 8-K/10-Q/10-K and the LLM only classifies the evidence, returning citations.

5. Rank. Pure arithmetic: `final = Σ(weight·score) × (0.70 + 0.30·confidence)`. Ranking = query fit only. Results ship with citations, limitations, and the funnel (100 → semantic → filtered → researched → qualified).

Why it's designed this way

Choice	Reason
Two-pass card generation (extract → verify)	Local 8B models hallucinate; the entailment check filters that out before anything is indexed.
Cards, not document chunks, for search	Atomic facts embed cleanly, match concepts ("data-center exposure") instead of keywords, and each hit is instantly citable.
LLM never does math or ranking	Growth %, filters, and scores are deterministic Python — reproducible and auditable.
Everything carries a source excerpt + URL	Every claim in the UI traces back to a specific SEC filing passage.